Software Requirements Specification

Restaurant App MVP (Frontend Only, React Native)

CS375 Mobile Application Development — Assignment 1 — Fall 2026

1. Introduction

1.1 Purpose

This document lays out what the Restaurant App MVP needs to do. Built with React Native and Expo, it's a frontend-only prototype supporting two kinds of users: Customers and Restaurant Managers. It covers Version 1.0, the scope defined for Assignment 1 of the Mobile Application Development course, and is meant for the course instructor, the developer (the author), and anyone who picks up the codebase later.

1.2 Scope

What Version 1.0 covers:

- Customer account creation and login (mocked, no real backend authentication)
- Browsing a categorised restaurant menu with daily specials and availability status
- Searching and sorting menu items
- Adding items to a cart, applying promo codes, and adjusting quantities
- Choosing between Pickup, Delivery, or Table Reservation at the point of ordering
- Reserving a table for a chosen date, time slot, and party size, based on mock table availability
- Placing an order and tracking its status through a simulated preparation timeline
- Light/dark theme toggle and a profile screen showing account details
- A Manager Dashboard for managing incoming orders, reservations, and menu items

What's left out for now:

- Real payment processing (credit card, wallets, or cash-on-delivery integration)
- Push notifications
- Any backend server, REST API, or live database — all data is mock JSON/JS held in React state and AsyncStorage
- Real-time GPS delivery tracking
- Multi-restaurant support (the MVP models a single restaurant)

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification — this document, defining what the system must do.
MVP	Minimum Viable Product — the smallest working version of the app that delivers core value.
UML	Unified Modeling Language — diagram notation used to visualise system structure and behaviour.
Hook	A React function (e.g. useState, useEffect) that lets a functional component use state or lifecycle features.
Context	React's built-in mechanism for sharing state across components without passing

	props manually at every level.
Reducer	A pure function that takes the current state and an action, and returns a new state (used with useReducer).
Mock Data	Hardcoded sample data (JSON/JS objects) used in place of a real backend/database.
FlatList	A React Native component that efficiently renders large scrollable lists by only rendering visible items.
AsyncStorage	A React Native library for persisting simple key-value data locally on the device.
Debounce	A technique that delays running a function until a pause in triggering events (e.g. typing) has occurred.

2. Overall Description

2.1 User Roles

Role	Key Permissions & Goals
Customer	Browse and search the menu, manage a cart, apply promo codes, reserve a table, place Pickup/Delivery/Dine-in orders, track order status in real time, and manage their own profile and theme preference.
Restaurant Manager	View and update the status of incoming orders, accept or decline table reservations, and add, edit, or toggle availability of menu items, with changes reflected instantly on the customer-facing menu.

2.2 User Stories

The following user stories were gathered from a self-interview as a prospective customer, supplemented by manager-side stories reflecting the restaurant owner's needs described in the assignment brief.

1. As a customer, I want to choose between Pickup, Delivery, or Table Reservation as soon as I open the app, so that I land directly in the flow that matches what I need.
2. As a customer, I want to see live table availability when reserving, so that I know a table is actually free before I show up.
3. As a customer, I want to see live pickup/delivery slot availability, so that I can pick a realistic time for my order.
4. As a customer, I want every button and screen to respond correctly and immediately to what I do, so that I trust the app enough to keep using it.
5. As a customer, I want to see the exact remaining time until my order is ready, so that I know when to arrive or expect delivery.
6. As a customer, I want to search the menu by name or category, so that I can quickly find a specific dish without scrolling through the whole menu.
7. As a customer, I want to apply a promo code at checkout, so that I can get a discount on eligible orders.
8. As a restaurant manager, I want to see incoming orders update live, so that I can prepare them without a phone ringing off the hook.
9. As a restaurant manager, I want to accept or decline table reservations, so that I can manage seating capacity effectively.
10. As a restaurant manager, I want to edit menu items and prices instantly, so that changes reflect on the customer app without needing a developer.

3. Functional Requirements

Each requirement below is independently testable and phrased as "The system shall...", grouped by module.

Authentication

FR-01: The system shall allow a new user to sign up as either a Customer or a Manager, providing full name, email, password, and confirm password.

FR-02: The system shall validate that the email is in a correct format and the password is at least 8 characters and contains at least one digit before allowing signup.

FR-03: The system shall allow an existing user to log in with email and password validated against a mock users array.

FR-04: The system shall navigate a Customer to the Menu screen and a Manager to the Manager Dashboard immediately after a successful login.

FR-05: The system shall display a clear inline error message when login or signup fails, without crashing or freezing the UI.

Menu

FR-06: The system shall load the menu from mock data and display a loading indicator while it is being fetched.

FR-07: The system shall display each menu item's name, description, price, category, image, and a "Daily Special" badge when applicable.

FR-08: The system shall grey out and disable the Add button for any menu item marked as unavailable.

FR-09: The system shall allow the customer to filter the menu by category using horizontal category chips.

FR-10: The system shall allow the customer to refresh the menu list using pull-to-refresh.

Search

FR-11: The system shall allow the customer to search menu items by name using a search bar with a debounce of 400 milliseconds.

FR-12: The system shall allow the customer to clear the search text and refocus the input with a single tap on a clear button.

FR-13: The system shall display a friendly empty-state message when no menu item matches the current search text.

FR-14: The system shall allow the customer to sort menu results by price (low to high, high to low) or name (A to Z).

Cart

FR-15: The system shall allow the customer to add a menu item to the cart, increment or decrement its quantity, and remove it entirely.

FR-16: The system shall allow the customer to attach a special instruction note (for example "no onions") to each cart item.

FR-17: The system shall allow the customer to apply a valid mock promo code to receive a percentage discount, and reject invalid codes with an error message.

FR-18: The system shall display a live item-count badge on the cart tab icon reflecting the current number of items in the cart.

FR-19: The system shall calculate and display subtotal, service charge, sales tax, promo discount, and grand total on the Order Summary screen.

Reservation

FR-20: The system shall allow the customer to select a reservation date, hourly time slot between 12:00 and 22:00, and party size between 1 and 12.

FR-21: The system shall disable a time slot when no table with sufficient seats is free at that time.

FR-22: The system shall validate that the reservation date is not in the past and that the booking is made at least one hour ahead of the selected time slot.

FR-23: The system shall validate that the contact phone number matches the Pakistani mobile format (03XX-XXXXXXX).

FR-24: The system shall show a confirmation modal summarising the reservation before it is saved, and allow the customer to cancel a reservation after an Alert confirmation.

Orders

FR-25: The system shall allow the customer to choose Dine-in (with a selected table) or Takeaway (with a pickup time) before placing an order.

FR-26: The system shall create a new order with a unique id, item list, total, type, status of Pending, and a timestamp when the customer places an order.

FR-27: The system shall automatically progress an order's status from Pending to Preparing, Ready, and Served on a fixed demo timer, and display a step progress indicator with an elapsed time counter.

FR-28: The system shall persist all placed orders locally using AsyncStorage so that they survive an app restart.

Manager Dashboard

FR-29: The system shall restrict access to the Manager Dashboard to users whose role is Manager.

FR-30: The system shall allow the manager to view all incoming orders and manually change an order's status.

FR-31: The system shall allow the manager to accept or decline a pending reservation.

FR-32: The system shall allow the manager to add a new menu item, edit an existing item's price, and toggle its availability, with the change reflected immediately on the customer's Menu screen through shared state.

4. Non-Functional Requirements

Usability

- The app must use clear, restaurant-familiar iconography (fork/knife, clock, table) so a first-time user needs no onboarding tutorial.
- Form validation errors must appear next to the specific field in error and clear as soon as the user edits that field.

Performance

- The Menu screen's initial mock data fetch must resolve within 1.5–2 seconds and show a loading indicator throughout.
- Scrolling a FlatList of 15+ menu items must remain smooth (no visible jank) by using keyExtractor and avoiding unnecessary re-renders via React.memo and useCallback.

Responsiveness

- All screens must render correctly on common phone screen sizes (small, standard, and large/tablet widths) using relative units and Flexbox layout rather than fixed pixel values.

Maintainability

- Source code must be organised under src/ into components, screens, context, reducers, hooks, data, and navigation folders, exactly matching the submission requirement.
- Reusable UI (e.g. the menu card) must be extracted into its own component rather than duplicated across screens.

Data Handling (No Backend)

- All application data (users, menu, tables, reservations, orders) must be defined as local JSON/JS files or generated mock arrays, held in React state, and where noted, persisted via AsyncStorage.
- The app must never make a network request to a real backend; any "loading" behaviour is simulated with setTimeout/Promise delays.

5. Client-Side Data Model (Mock Data)

Because no database is used, the following data sets are stored as local JSON/JavaScript files and held in React state (or AsyncStorage, for persistence across app restarts).

Data Set	Fields	Description
Users	id, fullName, email, password, role (Customer | Manager)	Mock account records in src/data/users.js used to validate login and signup.
Categories	id, name	The menu groupings — Starters, Mains, Desserts, Drinks — used to populate category chips.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	The restaurant's menu catalogue in src/data/menu.js, at least 15 items across 4+ categories.
Tables	id, seats, isAvailable	Physical tables in the restaurant, used to check reservation availability by party size.
Reservations	id, customerId, date, timeSlot, partySize, tableId, contactPhone, status	Customer table bookings, checked against Tables to disable unavailable slots.
Orders	id, customerId, items, total, type (Dine-in | Takeaway), status, timestamp	Placed orders, persisted via AsyncStorage, driving the Order Tracking and Manager Dashboard screens.

6. UML Diagrams

The five diagrams below, produced in Question 2, model the Restaurant App MVP from actor goals down to code-level structure.

6.1 Use Case Diagram

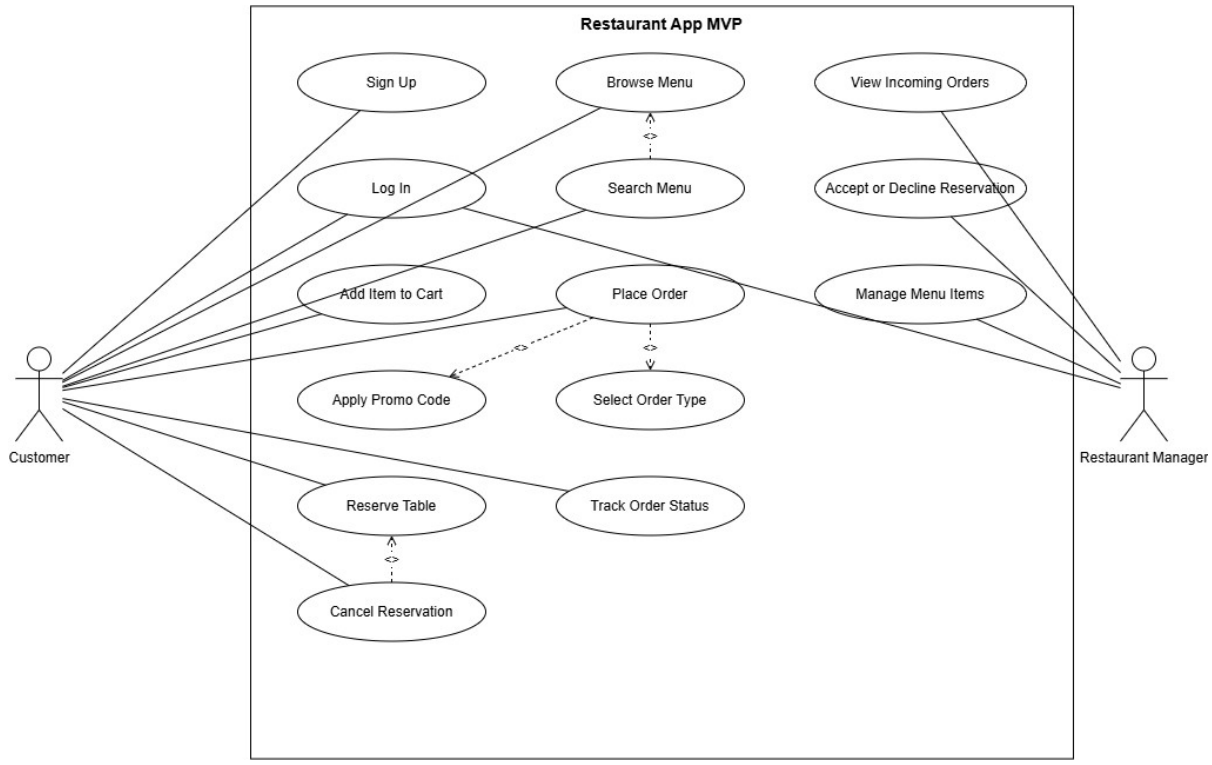


Figure 1: Use Case Diagram — Customer and Restaurant Manager actors and their use cases.

This diagram captures every interaction the two actors have with the system. The Customer's use cases span authentication, browsing, ordering, and reservations, while the Manager's use cases cover order fulfilment, reservation approval, and menu management. Place Order includes Apply Promo Code and Select Order Type, since both are mandatory sub-steps of placing an order; Cancel Reservation extends Reserve Table and Search Menu extends Browse Menu, since both are optional variations only available once the base use case is underway.

6.2 Class Diagram

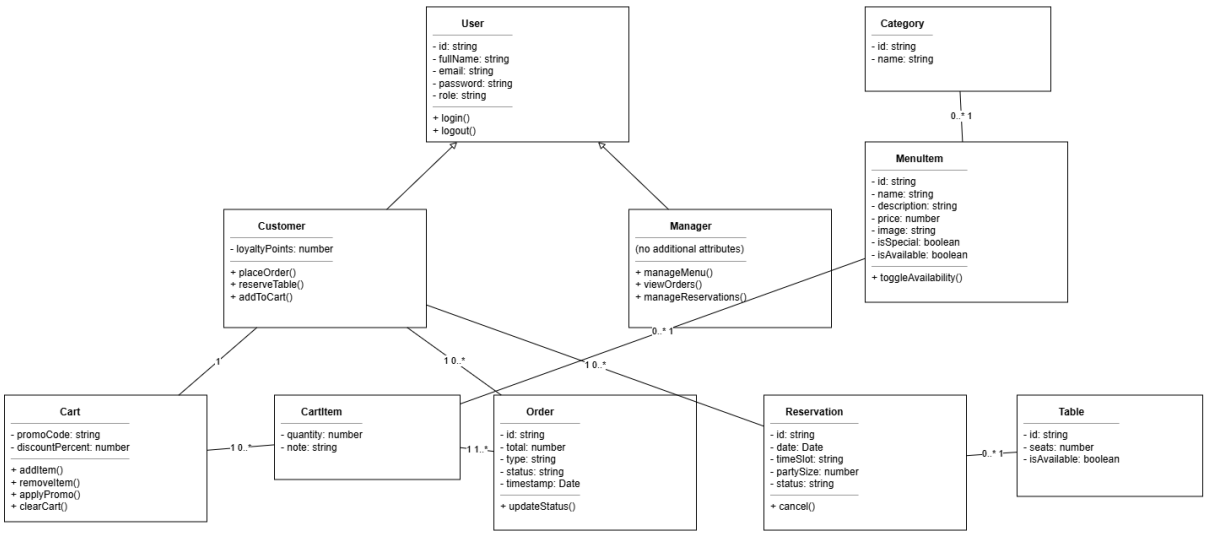


Figure 2: Class Diagram — User, Customer, Manager, Category, MenuItem, Cart, CartItem, Order, Reservation, and Table.

Customer and Manager both inherit from a shared User base class, avoiding duplicated authentication fields. A Customer owns exactly one Cart, which holds zero or more CartItems, each referencing one MenuItem; a MenuItem belongs to exactly one Category. A Customer may have many Orders and many Reservations, and each Reservation is tied to one Table. This structure maps directly onto the mock data model defined in Section 5.

6.3 Sequence Diagram

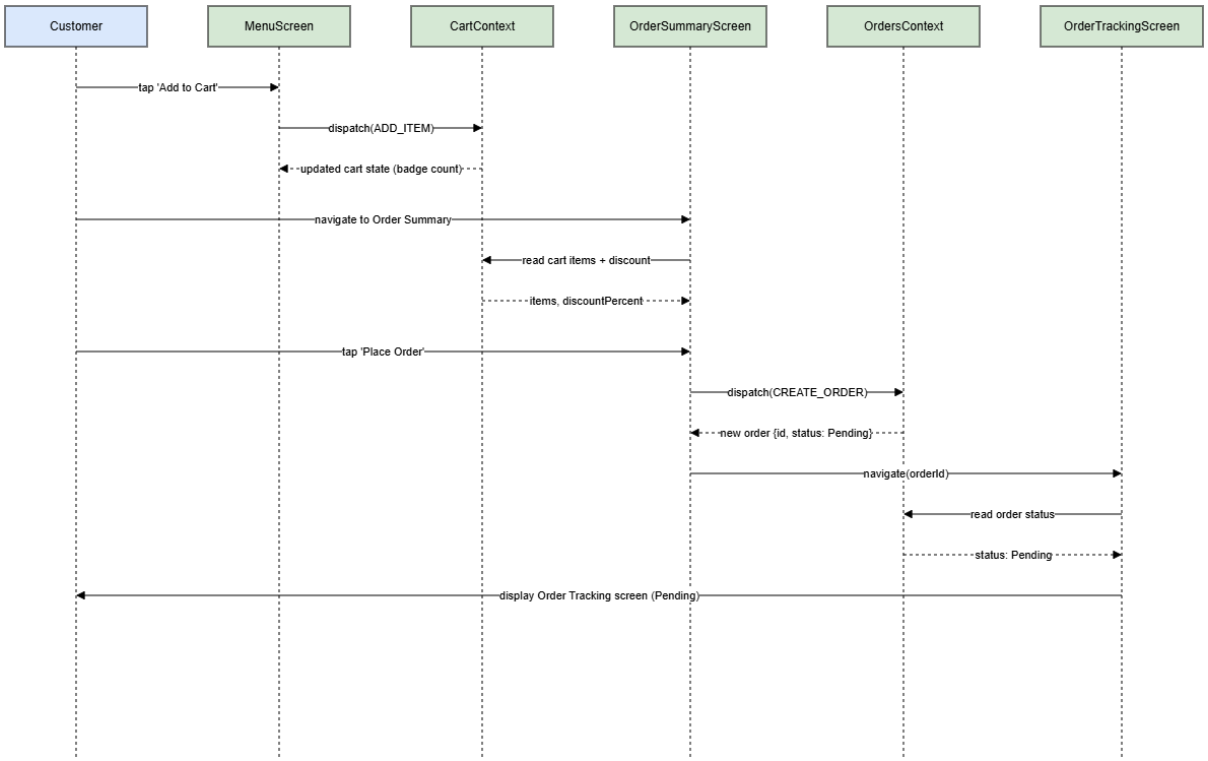


Figure 3: Sequence Diagram — "Customer places an order", from Add to Cart through to the Order Tracking screen.

This diagram traces the full path of a single order: tapping Add to Cart dispatches an action to CartContext, which updates the cart badge; navigating to Order Summary reads the cart items and discount back out of context; tapping Place Order dispatches a create-order action to OrdersContext, which returns a new order in Pending status; and the app navigates to Order Tracking, which reads and displays that status. This flow directly informed the useReducer design in Question 7 and the useEffect timer logic in Question 10.

6.4 State Machine Diagram

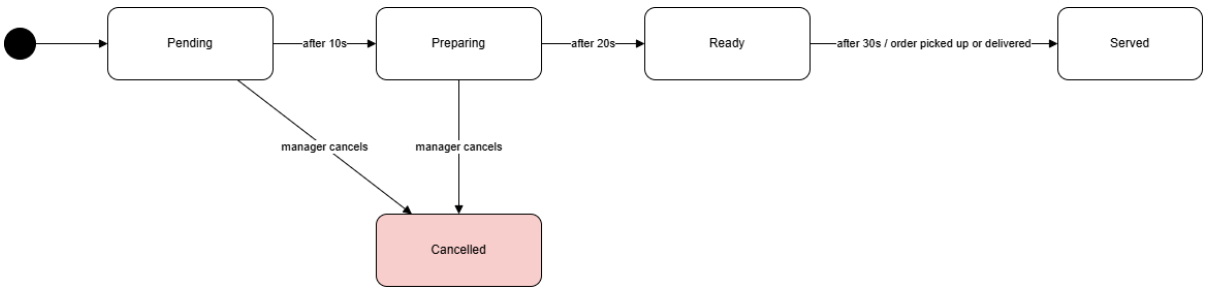


Figure 4: State Machine Diagram — the Order lifecycle.

An order begins in Pending and automatically advances to Preparing, then Ready, then Served on the demo timers specified in Question 10 (10, 20, and 30 seconds respectively). From either Pending or Preparing, the Manager may cancel the order, moving it to a terminal Cancelled state. No transition exists out of Served or Cancelled, since both represent the end of an order's lifecycle.

6.5 Component Diagram

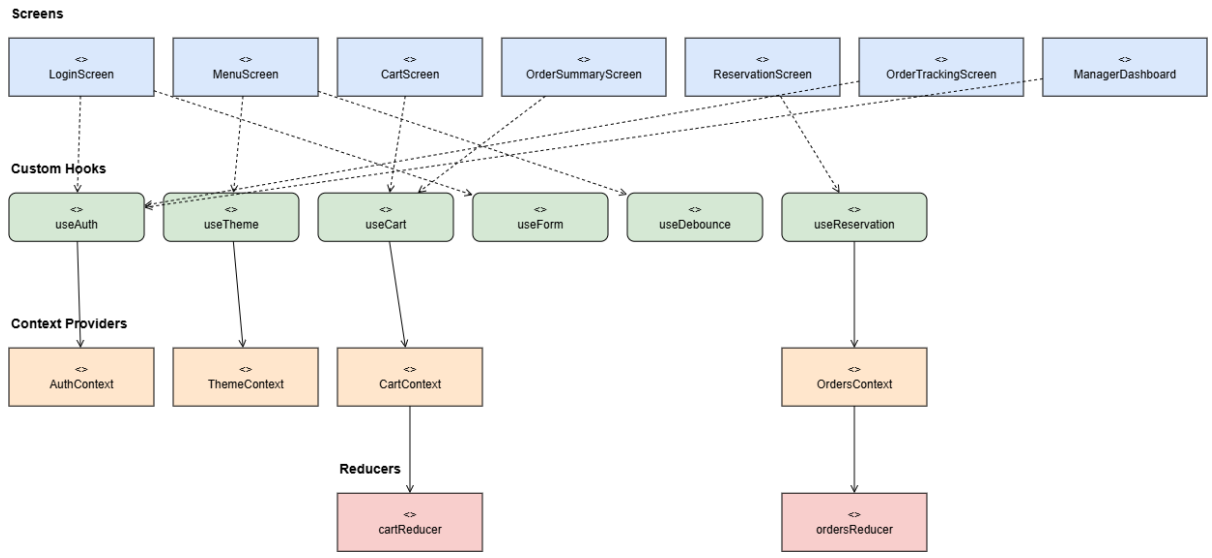


Figure 5: Component Diagram — screens, custom hooks, context providers, and reducers.

Screens never talk to Context directly; each screen calls a custom hook (useAuth, useTheme, useCart, useForm, useDebounce, or useReservation), and it is the hook that reads from the relevant Context Provider. AuthContext and ThemeContext hold their own state directly, while CartContext and OrdersContext are each backed by a dedicated reducer (cartReducer and ordersReducer) for more complex, multi-action state transitions. This layering keeps business logic out of the screens entirely, matching the "screen contains no business logic" requirement stated in Question 9.

7. MVP Frontend Development (React Native)

For each screen: purpose, UI elements, navigation entry/exit points, local data used, and the hooks used and why.

7.1 Login and Signup Screen

Purpose: Authenticate an existing user or register a new one, and route them to the correct home screen based on role.

UI Elements: Mode toggle (Login/Signup), email input, password input with show/hide toggle, full name and confirm-password and role selector (Signup only), submit button, ActivityIndicator while submitting.

Navigation: Entry point — app launch (no user logged in). Exit — Menu screen (Customer) or Manager Dashboard (Manager) on success; stays on screen with an Alert on failure.

Local Data Used: src/data/users.js (mock accounts).

Hooks Used: useState — for mode, form fields, errors object, show-password toggle, and isSubmitting flag, since each needs independent, re-render-triggering updates.

7.2 Menu Browsing Screen

Purpose: Let the customer browse, filter, and refresh the restaurant's menu.

UI Elements: Category chips, FlatList of menu cards (image, name, price, Daily Special badge), loading indicator, error message with Retry button, pull-to-refresh.

Navigation: Entry point — after login (Customer) or from any tab. Exit — Cart, Search, Profile, or Reservation screens via tabs/buttons.

Local Data Used: src/data/menu.js (mock menu, "fetched" via a delayed Promise).

Hooks Used: useEffect (empty dependency array) — to simulate the initial fetch and clean up its timer on unmount; a second useEffect depending on selectedCategory and menuItems — to derive filteredItems; useState — for menuItems, isLoading, error, and selectedCategory.

7.3 Search and Scroll Controls

Purpose: Let the customer quickly locate menu items and navigate a long list without excess scrolling.

UI Elements: Search bar with clear button, recent-search suggestion chips, floating "Back to top" button, small render-counter debug label.

Navigation: Lives inside the Menu screen; no separate route.

Local Data Used: The in-memory menuItems array and a small recent-searches array kept in state.

Hooks Used: useRef — for the TextInput focus ref, the debounce timeout id, the FlatList ref, the previous-query tracker, and the render counter, since none of these should trigger a re-render on their own.

7.4 Profile and Theme Screen

Purpose: Show the logged-in user's details and let them toggle light/dark mode.

UI Elements: Name, email, and role display; light/dark mode switch; logout button.

Navigation: Entry point — Profile tab. Exit — Login screen on logout (navigation stack reset).

Local Data Used: The current user object from AuthContext.

Hooks Used: useContext — via custom useAuth and useTheme hooks, so every screen can read the logged-in user and theme colours without prop drilling.

7.5 Cart Screen

Purpose: Let the customer review, adjust, and apply a promo code to items before ordering.

UI Elements: List of cart items with quantity stepper, remove button, special-instructions field per item, promo code input, apply/remove promo buttons.

Navigation: Entry point — Cart tab or "Add to Cart" flow. Exit — Order Summary screen.

Local Data Used: Cart state shaped as { items, promoCode, discountPercent }, plus a local mock promo-code object.

Hooks Used: useReducer — via CartContext and a useCart hook, since cart changes are multiple related, interdependent transitions (add, remove, increment, promo) best modelled as reducer actions rather than scattered useState calls.

7.6 Order Summary Screen

Purpose: Show a calculated bill before the order is placed, and demonstrate render-optimisation techniques.

UI Elements: Line-item list, subtotal, service charge, tax, discount, grand total, Place Order button.

Navigation: Entry point — from Cart screen. Exit — Order Tracking screen after placing the order.

Local Data Used: Cart items and discount from CartContext; named constants for tax/service rates.

Hooks Used: useMemo — to recompute totals only when cart items or discount change; useCallback — to keep card handlers (add to cart, toggle favourite) referentially stable so React.memo on MenuItemCard actually prevents unrelated re-renders.

7.7 Table Reservation Screen

Purpose: Let the customer book a table for a valid date, time, and party size with no business logic living in the screen itself.

UI Elements: Date picker, hourly time-slot chips (disabled when unavailable), party-size stepper, contact fields, confirmation modal, My Reservations list with cancel option.

Navigation: Entry point — Reservation tab or from the role-select landing choice. Exit — confirmation modal, then back to Menu or My Reservations.

Local Data Used: mockTables and mockReservations.

Hooks Used: Custom hooks — useForm (shared validation logic), useDebounce, and useReservation (availability checking, createReservation, cancelReservation), keeping the screen itself pure UI.

7.8 Order Tracking and Manager Dashboard

Purpose: Track a placed order's live status for the customer, and give the manager full control over orders, reservations, and menu.

UI Elements (Customer): Step progress indicator (Pending → Preparing → Ready → Served), running elapsed-time counter.

UI Elements (Manager): Three tabs — Incoming Orders (status buttons), Reservations (accept/decline), Menu Management (add/edit/toggle items).

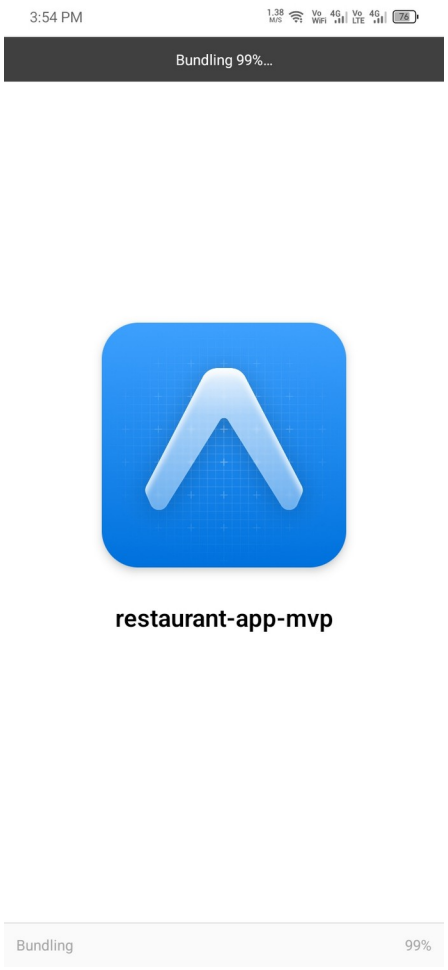
Navigation: Entry point — after placing an order (Customer) or login (Manager). Exit — back to Menu (Customer) or stays within Dashboard tabs (Manager).

Local Data Used: OrdersContext state, plus menu and reservation data shared with the customer-facing screens; all persisted via AsyncStorage.

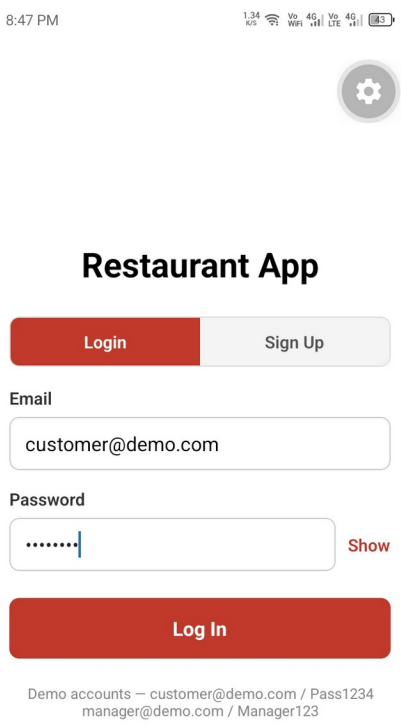
Hooks Used: useEffect with setInterval — to auto-advance order status on a demo timer and to update the elapsed-time counter every second, with cleanup on unmount; useEffect — to load/save AsyncStorage data on app start and on every relevant change; useReducer — via OrdersContext for order status transitions.

Appendix A: App Screenshots

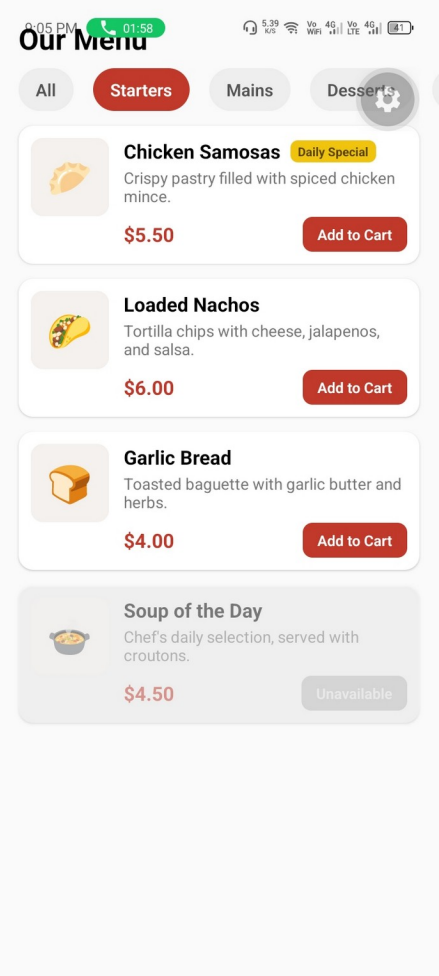
These screenshots come from the working MVP running live in Expo Go on a physical phone, matching the screens described in Section 7.



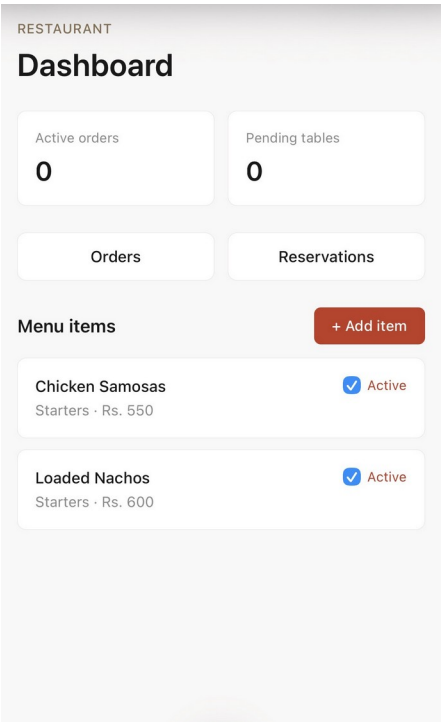
A.1: App loading in Expo Go (restaurant-app-mvp bundling).



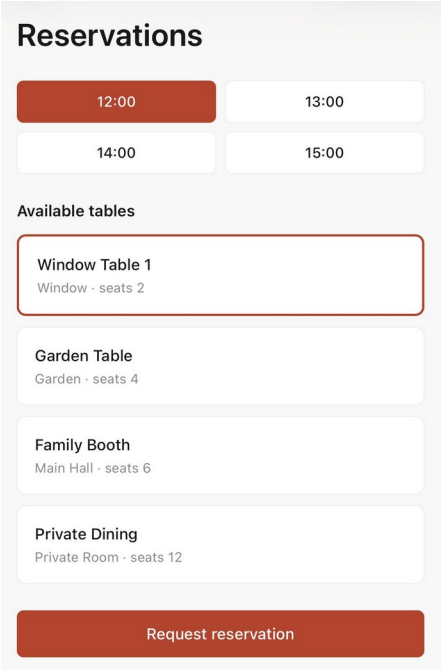
A.2: Login / Signup screen (Section 7.1) with demo account credentials shown.



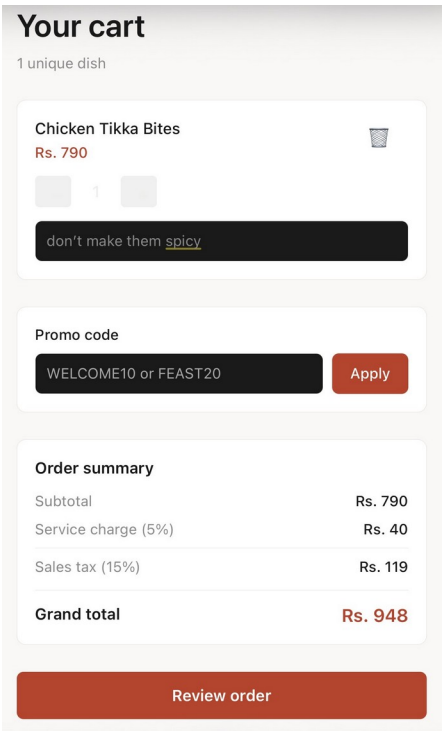
A.3: Menu Browsing screen (Section 7.2) filtered by the Starters category, showing a Daily Special badge and a disabled/unavailable item.



A.4: Manager Dashboard (Section 7.8) showing active orders, pending tables, and menu item management.



A.5: Table Reservation screen (Section 7.7) with time-slot selection and available tables.



A.6: Cart screen (Section 7.5) with special instructions, a promo code field, and the calculated Order Summary total.

— End of SRS —